

TrajCloak and TrajGuard: Stealthy Patch Attacks and Black-Box Trajectory Detection for Vision–Language–Action Models

Jiayang Pan and Chen Wan^(✉)

Department of Computer Science and Technology, Shantou University, Shantou,
China

23jypan@stu.edu.cn, wanchen18@outlook.com

Abstract. Vision–Language–Action (VLA) models directly map visual observations and language instructions to robot actions, making physical visual perturbations a safety-critical threat. Existing physical patch attacks often maximize task failure but induce conspicuous motion artifacts, such as large trajectory drift, abnormal spatial coverage, and high-frequency action oscillations. We propose TrajCloak, a trajectory-stealthy physical patch attack that combines representation-level disruption with trajectory deviation constraints, action smoothness regularization, adaptive constraint scheduling, and black–white–gray patch projection. These designs reduce both visual saliency and behavioral abnormality while preserving attack effectiveness. We further introduce TrajGuard, a black-box trajectory deviation detector that uses only action sequences and measures standardized deviations from clean behavior distributions with five motion-only features. Experiments on LIBERO with OpenVLA and SmolVLA show that TrajCloak maintains high Failure Rate while reducing Detection Score, Increase, and detection TPR under fixed FPR thresholds. These results suggest that VLA attacks should be evaluated jointly in terms of task disruption and behavioral detectability.

Keywords: Vision–Language–Action Models · Physical Patch Attack · Trajectory-Based Detection · Stealthiness · Black-Box Anomaly Detection

1 Introduction

Vision–Language–Action (VLA) models have emerged as a central paradigm for general-purpose robotic control, mapping visual observations and natural-language instructions directly to executable actions [2,5,16,11,8]. By unifying perception, language understanding, and action generation, these models enable robots to follow open-vocabulary instructions, interact with multi-object scenes, and perform long-horizon manipulation tasks. However, this end-to-end perception-to-action pipeline also exposes robotic execution to visual input vulnerabilities: perturbations in the observation space may propagate into unsafe end-effector motions, task failures, or collisions.

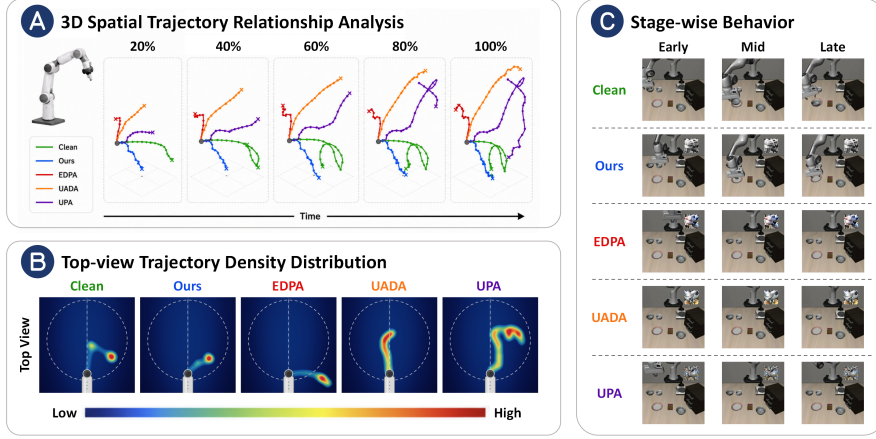


Fig. 1. Visualization of robot trajectories and stage-wise behavior under different physical patch attacks. Existing attacks tend to induce larger spatial dispersion and more visible motion anomalies, whereas TrajCloak keeps the trajectory distribution closer to clean executions while maintaining attack effectiveness.

Unlike attacks on conventional vision models, physical attacks against VLA models should not be evaluated solely by whether they reduce task success. Since VLA outputs are directly executed in the physical world, an effective attack may still be easily detected if it induces unnatural robot behavior. Existing physical patch attacks typically maximize action deviation or disrupt vision–language representation alignment [6, 1, 14, 15], but they rarely constrain the resulting action trajectory. Consequently, attacked robots may exhibit abnormal motion patterns, such as excessive spatial dispersion, frequent direction changes, high-frequency jitter, or large deviations from the task-relevant workspace. These behaviors can improve attack effectiveness but also make the attack more visible to humans or trajectory-based safety monitors.

Fig. 1 illustrates this motivation. Existing attacks can substantially alter robot executions, yet their trajectories often become more dispersed than clean behaviors. In contrast, a stealthy physical attack should remain inconspicuous in both appearance and execution: the patch should be visually unobtrusive, and the induced trajectory should stay close to normal behavior while still causing task-disruptive deviations at critical stages.

To this end, we present a unified attack-and-detection framework, as shown in Fig. 2. On the attack side, we propose TrajCloak, a trajectory-stealthy physical patch attack that combines representation-level disruption with trajectory deviation and smoothness constraints. An adaptive constraint schedule prevents over-constraining the patch before a meaningful attack signal emerges, while a black–white–gray projection restricts the patch to a low-color-complexity pattern to reduce visual saliency. On the detection side, we introduce TrajGuard,

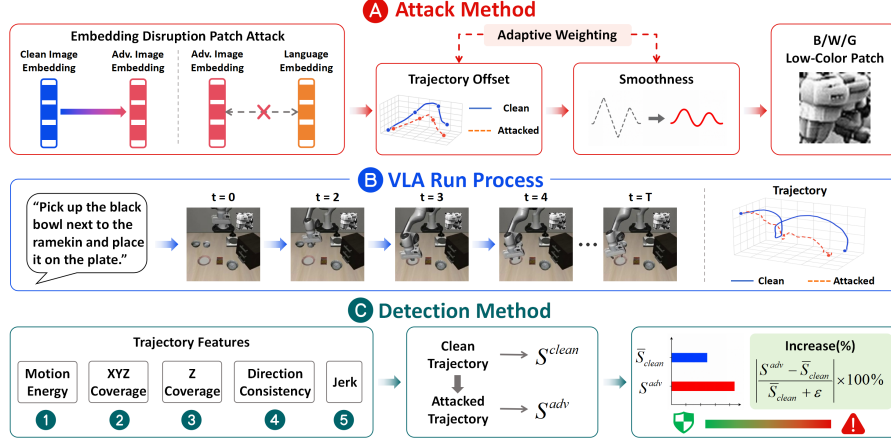


Fig. 2. Overview of TrajCloak and TrajGuard. The attack module generates a stealthy physical patch through representation-level attack, trajectory constraints, smoothness regularization, and black–white–gray projection. The detection module extracts trajectory statistics from action sequences and computes deviation scores with respect to the clean behavior distribution.

a black-box trajectory deviation detector that uses only action sequences. It extracts motion-only statistics and computes standardized deviation scores with respect to the clean behavior distribution, requiring no access to model parameters, gradients, or intermediate features.

Our main contributions are summarized as follows:

- We identify trajectory naturalness as an important stealthiness criterion for physical VLA attacks, in addition to patch appearance.
- We propose TrajCloak, a trajectory-stealthy physical patch attack that combines representation disruption, trajectory deviation control, action smoothness regularization, and low-color-complexity visual projection.
- We introduce TrajGuard, a black-box trajectory-based detector that models clean robot behaviors using motion-only features and standardized abnormality scores.
- Experiments on LIBERO with OpenVLA and SmolVLA show that TrajCloak maintains a high Failure Rate while reducing Detection Score and detection TPR, yielding a better balance between attack effectiveness and trajectory stealthiness.

2 Related Work

2.1 Security of Vision–Language–Action Models

Vision–Language–Action (VLA) models map multimodal observations and language instructions to executable robot actions, enabling strong generalization in open-ended manipulation tasks [2,5,16,11,8,12]. However, this tight perception-to-action coupling also introduces safety risks, as input perturbations can propagate into task-disruptive or unsafe robot behaviors [14,15]. Unlike conventional vision models, VLA security must consider not only perceptual robustness, but also control stability, trajectory plausibility, and task-level failure. This work studies physical patch attacks on VLA models from both attack effectiveness and trajectory-level detectability.

2.2 Physical Patch Attacks

Physical patch attacks manipulate model predictions by placing printable adversarial patterns in the scene, making them practical threats for embodied systems [7,6,1,13]. For VLA-controlled robots, such attacks must go beyond inducing recognition errors and account for action-token distributions, motion geometry, and execution outcomes.

Recent VLA-oriented attacks optimize patches in either the action or representation space. UADA maximizes the discrepancy between adversarial and reference action predictions, while UPA further leverages end-effector geometry to steer the predicted motion away from the intended trajectory [14]. These objectives can substantially increase task failure rates, but their explicit emphasis on enlarging action deviations may produce abnormal trajectories that are easier to detect. EDPA instead disrupts visual–language representation alignment and maximizes the discrepancy between clean and adversarial visual features, improving model-agnostic transferability [15]. However, EDPA does not explicitly optimize visual saliency or trajectory stealthiness.

In contrast, TrajCloak jointly considers attack effectiveness and stealthiness. It retains representation-level disruption, while introducing a black–white–gray low-color-complexity projection and trajectory deviation and smoothness constraints. As a result, TrajCloak treats trajectory naturalness and detectability as explicit optimization objectives rather than only post-hoc evaluation metrics.

2.3 Robot Trajectory Anomaly Detection

Robot execution monitoring often detects abnormal behaviors using motion statistics such as trajectory length, spatial extent, velocity variation, acceleration variation, and directional consistency [10,4,3]. Compared with detectors that require internal activations or gradients, trajectory-based detection is more suitable for black-box robotic deployment, where control commands and end-effector motions are typically observable. Following this setting, TrajGuard models the clean behavior distribution using motion-only trajectory features and computes

standardized deviation scores to quantify the detectability of physical patch attacks.

3 Method

Our framework consists of two modules: TrajCloak and TrajGuard. TrajCloak aims to generate a physical patch that can induce task failure without producing obvious abnormal motion. TrajGuard quantifies, under a black-box setting, how much an attacked trajectory deviates from the clean behavior distribution. Together, they form an attack-detection framework for studying physical security in VLA systems.

3.1 Problem Definition

Let the VLA policy be denoted by π_θ . Given an image observation I_t and a language instruction q , the model outputs an action $a_t = \pi_\theta(I_t, q)$. A physical patch δ is spatially transformed and blended into the scene to produce an attacked image $I_t^{adv} = \mathcal{A}(I_t, \delta)$, which induces the attacked action $a_t^{adv} = \pi_\theta(I_t^{adv}, q)$. Our objective is not to simply maximize the discrepancy between a_t^{adv} and the clean action a_t^{clean} . Instead, we seek to maintain a high task failure rate while keeping the attacked trajectory $\tau^{adv} = \{a_t^{adv}\}_{t=1}^T$ close to the normal trajectory distribution, thereby reducing behavioral detectability.

3.2 Representation-Level Attack Objective

We first retain the core idea of EDPA-style representation disruption [15]. Let h_I be the clean image representation, h_I^{adv} be the attacked image representation, and h_L be the language representation. The attack objective consists of an image-representation deviation term and a vision-language alignment disruption term:

$$\begin{aligned}\mathcal{L}_{patch} &= \text{CE}\left(\frac{\text{sim}(h_I^{adv}, h_I)}{\tau}, y\right), \\ \mathcal{L}_{align} &= \|\text{sim}(h_I, h_L) - \text{sim}(h_I^{adv}, h_L)\|_1, \\ \mathcal{L}_{attack} &= \alpha \mathcal{L}_{patch} + (1 - \alpha) \mathcal{L}_{align},\end{aligned}\tag{1}$$

where $\text{sim}(\cdot, \cdot)$ denotes cosine similarity, τ is the temperature, y represents the within-batch correspondence, and α controls the relative weight between the two terms. Maximizing $\mathcal{L}_{attack}$ increases the ability of the patch to disrupt VLA representations and action predictions.

3.3 Trajectory Deviation and Action Smoothness Constraints

Optimizing only the attack objective can lead to large end-effector deviations or high-frequency oscillations. To improve trajectory stealthiness, we introduce

two action-level constraints. Let $\{u_t^{clean}\}_{t=1}^T$ and $\{u_t^{adv}\}_{t=1}^T$ denote the clean and attacked main action components, respectively. The trajectory deviation constraint is defined as

$$\mathcal{L}_{traj} = \frac{1}{T} \sum_{t=1}^T [\max(\|u_t^{adv} - u_t^{clean}\|_2 - m_\tau, 0)]^2, \quad (2)$$

where m_τ is the allowable action-deviation margin. This term penalizes only deviations beyond a tolerable range, avoiding unnecessary suppression of task-disruptive perturbations.

We further characterize local smoothness using second-order action differences:

$$\begin{aligned} \Delta^2 u_t &= u_t - 2u_{t-1} + u_{t-2}, \\ \mathcal{L}_{smooth} &= \frac{1}{T-2} \sum_{t=3}^T [\max(\|\Delta^2 u_t^{adv}\|_2 - \|\Delta^2 u_t^{clean}\|_2 - m_s, 0)]^2. \end{aligned} \quad (3)$$

This constraint suppresses additional abrupt action variations introduced by the attack, reducing local jitter and sudden direction changes.

3.4 Adaptive Constraint Weighting

During early optimization, overly strong trajectory constraints can prevent the patch from forming an effective attack signal. Conversely, once the attack becomes effective, unconstrained optimization can produce conspicuously abnormal trajectories. We therefore dynamically adjust the constraint weights according to the current attack strength A :

$$\begin{aligned} g(A) &= \text{clip}\left(\frac{A - A_{low}}{A_{high} - A_{low} + \epsilon}, 0, 1\right), \\ \lambda_\tau^{eff} &= \lambda_\tau g(A), \quad \lambda_s^{eff} = \lambda_s g(A). \end{aligned} \quad (4)$$

When the attack is weak, $g(A)$ remains small and the optimization focuses primarily on the attack objective. As the attack strength increases, trajectory and smoothness constraints are gradually activated, enabling a dynamic balance between attack effectiveness and trajectory stealthiness.

3.5 Black–White–Gray Low-Color-Complexity Patch

To reduce the visual saliency of the patch, we restrict it to a ternary black, gray, and white color space. The patch is initialized as

$$\delta(i, j) \in \{0, 0.5, 1\}. \quad (5)$$

During optimization, we jointly control the patch appearance using grayscale consistency, ternary quantization, and total variation regularization:

$$\begin{aligned}
\mathcal{L}_{gray} &= \frac{1}{3} (\|\delta_R - \delta_G\|_2^2 + \|\delta_R - \delta_B\|_2^2 + \|\delta_G - \delta_B\|_2^2), \\
\bar{\delta} &= \frac{1}{3} (\delta_R + \delta_G + \delta_B), \\
\mathcal{L}_{quant} &= \frac{1}{HW} \sum_{i,j} \min_{c \in \{0, 0.5, 1\}} (\bar{\delta}_{i,j} - c)^2, \\
\mathcal{L}_{tv} &= \frac{1}{HW} \sum_{i,j} (|\delta_{i+1,j} - \delta_{i,j}| + |\delta_{i,j+1} - \delta_{i,j}|), \\
\mathcal{L}_{visual} &= \lambda_{gray} \mathcal{L}_{gray} + \lambda_{quant} \mathcal{L}_{quant} + \lambda_{tv} \mathcal{L}_{tv}.
\end{aligned} \tag{6}$$

Here, $\mathcal{L}_{gray}$ encourages consistency across RGB channels, $\mathcal{L}_{quant}$ moves pixels toward the black, gray, and white centers, and $\mathcal{L}_{tv}$ reduces local high-frequency noise. To further preserve grayscale appearance, we average the gradients across the three channels before updating the patch:

$$\nabla_{\delta}^{gray} = \frac{\nabla_{\delta_R} + \nabla_{\delta_G} + \nabla_{\delta_B}}{3}. \tag{7}$$

3.6 Overall Optimization Objective

Combining the above objectives, we optimize

$$\max_{\delta} \mathcal{J}(\delta) = \mathcal{L}_{attack} - \lambda_r^{eff} \mathcal{L}_{traj} - \lambda_s^{eff} \mathcal{L}_{smooth} - \lambda_v \mathcal{L}_{visual}. \tag{8}$$

The patch is updated by signed gradient ascent:

$$\delta^{k+1} = \Pi_{[0,1]} (\delta^k + \eta \cdot \text{sign}(\nabla_{\delta} \mathcal{J}(\delta^k))), \tag{9}$$

followed by a soft ternary projection:

$$\delta \leftarrow (1 - \rho) \delta + \rho Q_{BWG}(\delta), \tag{10}$$

where $Q_{BWG}(\cdot)$ maps each value to its nearest black, gray, or white center. We use soft projection during training to preserve optimization flexibility and apply hard projection when saving the final patch to obtain a strict ternary pattern.

3.7 TrajGuard: Trajectory Deviation Detection from Action Sequences

To assess whether an attacked trajectory is abnormal, TrajGuard defines a trajectory deviation score that does not rely on any internal model state. Given an action sequence

$$A = \{a_1, a_2, \dots, a_T\}, \quad a_t = [\Delta x_t, \Delta y_t, \Delta z_t, \Delta r_t, \Delta \theta_t, \Delta \psi_t, g_t], \tag{11}$$

we take the first three translational components as $\Delta p_t = [\Delta x_t, \Delta y_t, \Delta z_t]$ and reconstruct the relative trajectory as $p_t = \sum_{i=1}^t \Delta p_i$. We extract five motion-only features from each trajectory:

$$F(\tau) = [f_1, f_2, f_3, f_4, f_5], \quad (12)$$

where

$$\begin{aligned} f_1 &= \sum_{t=1}^T \|\Delta p_t\|_2, \\ f_2 &= (\max_t x_t - \min_t x_t) + (\max_t y_t - \min_t y_t) + (\max_t z_t - \min_t z_t), \\ f_3 &= \max_t z_t - \min_t z_t, \\ f_4 &= \frac{1}{|\mathcal{I}|} \sum_{t \in \mathcal{I}} \frac{\Delta p_t \cdot \Delta p_{t-1}}{\|\Delta p_t\|_2 \|\Delta p_{t-1}\|_2}, \\ f_5 &= \frac{1}{T-2} \sum_{t=3}^T \|\Delta p_t - 2\Delta p_{t-1} + \Delta p_{t-2}\|_2. \end{aligned} \quad (13)$$

These five features describe accumulated motion magnitude, three-dimensional spatial coverage, vertical displacement range, directional consistency between consecutive actions, and higher-order action smoothness, respectively. When computing f_4 , we skip pairs whose adjacent action norms are close to zero to avoid numerical instability.

During clean distribution modeling, we extract features F_i^{clean} from N clean trajectories and compute the mean and standard deviation for each feature dimension:

$$\mu_j = \frac{1}{N} \sum_{i=1}^N F_{i,j}^{clean}, \quad \sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^N (F_{i,j}^{clean} - \mu_j)^2 + \epsilon}. \quad (14)$$

For both clean and attacked trajectories, we compute standardized deviation scores as

$$z_{i,j} = \left| \frac{F_{i,j} - \mu_j}{\sigma_j} \right|, \quad S_i = \frac{1}{5} \sum_{j=1}^5 z_{i,j}. \quad (15)$$

Let $\bar{S}_{clean}$ denote the average natural fluctuation of clean trajectories. The relative deviation increase of an attacked trajectory is then

$$R_k = \left| \frac{S_k^{adv} - \bar{S}_{clean}}{\bar{S}_{clean} + \epsilon} \right| \times 100\%. \quad (16)$$

In TrajGuard, a lower Increase indicates less additional trajectory abnormality introduced by the attack.

4 Experiments

4.1 Experimental Setup

Datasets and tasks. We evaluate on the LIBERO simulation benchmark, covering four task suites: Spatial, Object, Goal, and Long [9]. Spatial mainly evaluates spatial-relation understanding, Object emphasizes target-object recognition, Goal focuses on achieving desired target states, and Long includes longer-horizon multi-stage manipulation procedures. Together, these suites cover representative capability requirements for VLA models across embodied manipulation scenarios.

Victim models. We evaluate two VLA models, OpenVLA and SmolVLA [8,12]. OpenVLA is used for the full comparison among Clean, UADA, UPA, EDPA, and TrajCloak. SmolVLA is used to further examine whether TrajCloak generalizes to another VLA architecture. For UADA and UPA results that are unavailable on SmolVLA, we report “—” in the table.

Compared methods. Clean denotes the no-attack setting and provides the baseline clean trajectory distribution and task failure rate. UADA, UPA, and EDPA are existing physical patch attack baselines [14,15]. Our attack method, TrajCloak, aims to preserve strong task-disruption capability while reducing detectability at both the visual and trajectory levels. Our detection method, TrajGuard, evaluates the deviation of attacked trajectories from the clean behavior distribution under a black-box setting.

Implementation details. All patches are set to 50×50 pixels, and the input image resolution is 224×224 . The patch update step size is $2/255$, the maximum number of optimization iterations is 50,000, and the batch size is 4. TrajCloak finally applies hard black–white–gray projection to obtain a low-color-complexity patch.

Evaluation metrics. We evaluate on the four LIBERO task suites. Each task suite contains 10 tasks, and each task is independently executed 50 times; therefore, each suite includes 500 evaluation rollouts. Attack effectiveness is measured by Failure Rate (FR), defined as the fraction of executions that fail to complete the task, equivalently $FR = 1 - SR$, where SR denotes success rate. A higher FR indicates a more effective attack. Trajectory stealthiness is evaluated using Detection Score (DS) and Increase. DS is computed by TrajGuard as the standardized trajectory deviation from the clean behavior distribution, while Increase measures the relative growth of the detection score compared with clean trajectories. Lower DS and Increase indicate that the attacked trajectory is closer to the clean behavior distribution and therefore more trajectory-stealthy.

4.2 Main Results

Table 1 summarizes trajectory stealthiness and attack effectiveness for different attack methods on OpenVLA and SmolVLA. Existing baselines generally increase Failure Rate substantially, but they also amplify Detection Score. For example, on the Spatial suite of OpenVLA, the DS values of UADA, UPA, and

Table 1. Results on LIBERO. Lower DS/Increase indicates stronger trajectory stealthiness, while higher FR indicates stronger attack effectiveness.

Task Suite	Method	OpenVLA			SmolVLA		
		DS	Increase↓	FR↑	DS	Increase↓	FR↑
Spatial	Clean	0.7598	—	14.1±0.5%	0.6708	—	10.20%
	UADA	2.2748	199.38%	98.9±0.1%	—	—	—
	UPA	2.3377	207.66%	99.1±0.3%	—	—	—
	EDPA	2.2036	190.01%	100±0.0%	1.9473	190.30%	46.20%
	TrajCloak	1.9381	155.07%	100±0.0%	1.4000	108.70%	39.60%
Object	Clean	0.6585	—	12.0±0.4%	0.7119	—	10.40%
	UADA	1.6622	152.44%	92.5±0.7%	—	—	—
	UPA	1.7699	168.79%	92.1±0.8%	—	—	—
	EDPA	2.0664	213.82%	100±0.0%	2.1729	205.24%	38.40%
	TrajCloak	1.7728	169.23%	100±0.0%	1.9300	171.12%	34.40%
Goal	Clean	0.7084	—	26.9±1.5%	0.7245	—	12.20%
	UADA	1.2972	83.11%	98.6±0.1%	—	—	—
	UPA	1.4081	98.76%	98.9±0.2%	—	—	—
	EDPA	1.3263	87.21%	100±0.0%	1.8255	151.98%	54.60%
	TrajCloak	1.3256	87.12%	100±0.0%	1.3697	89.07%	48.20%
Long	Clean	0.7172	—	48.1±1.9%	0.7296	—	22.60%
	UADA	2.0498	185.82%	99.6±0.2%	—	—	—
	UPA	2.0089	180.12%	99.6±0.3%	—	—	—
	EDPA	2.1773	203.60%	100±0.0%	2.1022	188.12%	58.00%
	TrajCloak	1.8291	155.05%	100±0.0%	1.4495	98.66%	52.80%

EDPA are 2.2748, 2.3377, and 2.2036, respectively, with Increase values close to or above 190%. This indicates that these methods are highly disruptive but also induce trajectories that deviate substantially from the clean distribution.

In contrast, TrajCloak achieves lower trajectory deviation scores across multiple task suites while maintaining high task failure rates. On OpenVLA, TrajCloak obtains the lowest or near-lowest DS on Spatial, Goal, and Long, and maintains 100% FR across all four suites. On SmolVLA, TrajCloak consistently reduces DS and Increase compared with EDPA. For instance, on Long, DS decreases from 2.1022 to 1.4495, and Increase decreases from 188.12% to 98.66%. These results show that TrajCloak better balances task disruption and trajectory stealthiness by explicitly suppressing abnormal trajectory patterns while preserving attack effectiveness.

4.3 Trajectory Detectability under Fixed False-Positive Rates

Table 2 reports the True Positive Rate (TPR) on OpenVLA under fixed False Positive Rate (FPR) thresholds. Here, FPR denotes the proportion of clean trajectories that are incorrectly classified as attacked. Fixing an FPR threshold is equivalent to setting a detection threshold on the clean distribution to control the probability of false alarms during normal executions. Under this threshold, TPR measures the proportion of attacked trajectories that are successfully detected. Since TrajGuard is based on trajectory deviation scores, a higher TPR indicates that attacked trajectories are easier to distinguish from clean trajectories, whereas a lower TPR indicates stronger stealthiness.

Table 2. Trajectory detection TPR of TrajGuard on OpenVLA under different FPR thresholds. Lower TPR indicates that the attack is harder to detect.

Task Suite	Method	TPR		
		FPR=1%	FPR=5%	FPR=10%
Spatial	UADA	60.40%	70.60%	84.80%
	UPA	64.20%	76.60%	94.80%
	EDPA	60.20%	72.80%	86.80%
	TrajCloak	46.60%	54.80%	72.60%
Object	UADA	4.80%	32.80%	78.80%
	UPA	14.80%	40.80%	82.80%
	EDPA	20.40%	52.40%	80.40%
	TrajCloak	12.80%	34.80%	74.80%
Goal	UADA	2.40%	6.40%	34.40%
	UPA	8.20%	22.40%	38.40%
	EDPA	2.60%	20.40%	54.20%
	TrajCloak	0.80%	10.20%	36.20%
Long	UADA	30.20%	72.20%	88.80%
	UPA	36.40%	66.80%	82.20%
	EDPA	38.40%	66.40%	80.80%
	TrajCloak	14.40%	54.40%	76.80%

The results show that TrajCloak achieves lower TPR under most task suites and FPR thresholds. For example, on Long, when FPR is fixed at 1%, the TPR of TrajCloak is 14.40%, substantially lower than those of UADA, UPA, and EDPA. On Spatial, when FPR is 5%, TrajCloak reaches a TPR of 54.80%, also lower than all three baselines. These results further validate, from the perspective of detection separability, that trajectory constraints improve attack stealthiness.

Table 3. Effect of different TrajCloak components on trajectory stealthiness and attack effectiveness. The experiment is based on the EDPA baseline in the Spatial suite.

Method Variant	Gray	L_{traj}	L_{smooth}	Weight	DS	Increase↓	FR↑
EDPA baseline	×	×	×	×	2.2036	190.01%	100.00%
+ Gray	✓	×	×	×	2.1949	188.86%	100.00%
+ L_{traj}	✓	✓	×	×	2.1858	187.67%	98.84%
+ L_{smooth}	✓	✓	✓	×	1.9299	153.99%	98.16%
+ Adaptive Weight	✓	✓	✓	✓	1.9381	155.07%	100.00%

4.4 Ablation Study

Table 3 analyzes the contribution of each component in TrajCloak. The EDPA baseline denotes the base attack without the proposed constraints. Adding only the grayscale constraint reduces DS from 2.2036 to 2.1949, indicating that the

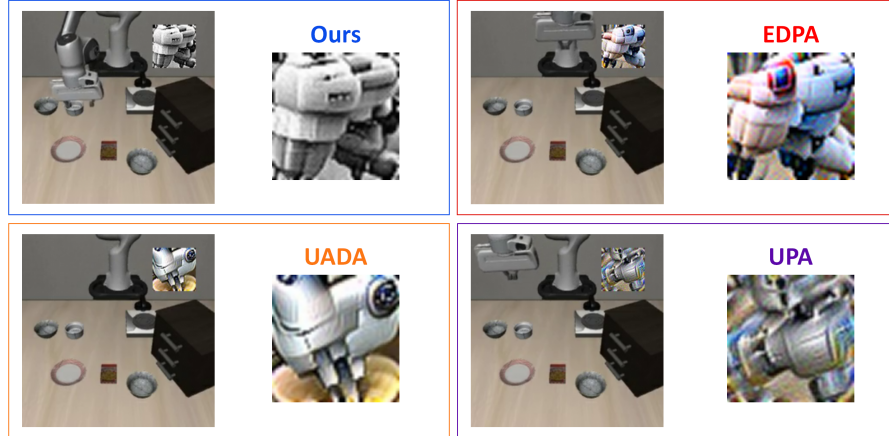


Fig. 3. Visual comparison of physical patches generated by different attack methods. EDPA, UADA, and UPA tend to contain more salient colored textures, whereas TrajCloak restricts the patch to the black–white–gray space to reduce color complexity and visual saliency.

low-color-complexity constraint does not significantly weaken the attack but has limited direct impact on trajectory abnormality. Adding $\mathcal{L}_{traj}$ further reduces DS to 2.1858, showing that trajectory deviation constraints can suppress excessive action offsets. Incorporating $\mathcal{L}_{smooth}$ substantially lowers DS to 1.9299 and reduces Increase to 153.99%, suggesting that high-frequency jitter and abrupt action variations are important sources of trajectory abnormality. Finally, adaptive weighting restores FR to 100.00% while maintaining low Increase, confirming the need to dynamically balance attack strength and trajectory stealthiness.

4.5 Patch Appearance and Trajectory Feature Analysis

Fig. 3 shows the patches generated by different methods. The baselines typically contain more salient colored textures or local high-frequency patterns, which can help disrupt visual representations but also increase visual saliency in physical scenes. TrajCloak restricts the patch to the black–white–gray space and therefore substantially reduces color complexity. Combined with Table 1, the results show that TrajCloak still maintains near or exact 100% FR on OpenVLA even under the color-space constraint, indicating that the visual-stealthiness constraint does not eliminate the main attack capability.

Table 4 further reports standardized deviation scores for the five trajectory features. TrajCloak achieves the lowest values on f_1 , f_2 , f_3 , and f_5 , indicating that it reduces abnormal accumulated motion, three-dimensional spatial coverage, vertical range, and second-order action variation. Although UADA obtains the lowest value on the directional consistency feature f_4 , it deviates substantially on other

Table 4. Standardized deviation scores of the five trajectory features extracted by TrajGuard. All features are normalized with respect to the clean behavior distribution, and lower values indicate trajectories closer to normal executions.

Method	$f_1 \downarrow$	$f_2 \downarrow$	$f_3 \downarrow$	$f_4 \downarrow$	$f_5 \downarrow$
UADA	1.974	3.204	3.367	0.695	2.091
UPA	2.136	3.267	2.543	1.786	1.823
EDPA	2.483	3.211	1.981	1.484	1.813
TrajCloak	1.821	2.548	1.869	1.756	1.696

features. This suggests that a single motion indicator is insufficient to fully characterize trajectory naturalness. By jointly modeling five complementary features, TrajGuard provides a more stable assessment of trajectory abnormality.

5 Discussion

Our results show that task failure rate alone is insufficient for evaluating physical attacks on VLA models. A high-FR attack may still be easy to detect if it induces visibly abnormal robot motions. Trajectory-stealthy attacks therefore need to balance task disruption with behavioral naturalness. TrajCloak addresses this trade-off through adaptive constraints: it first emphasizes representation-level disruption to establish attack effectiveness, and then increasingly suppresses abnormal motion as the attack signal becomes stronger.

TrajGuard offers a deployment-friendly monitoring mechanism, as it requires only recorded action sequences and no access to model weights or intermediate features. This makes it applicable to closed-source, cloud-hosted, or third-party VLA systems. Its reliability, however, depends on the coverage of the clean behavior distribution; rare but valid executions may be misclassified if normal trajectory diversity is underrepresented. Future work may improve robustness by incorporating task-stage modeling, temporal probabilistic models, or uncertainty estimation.

6 Conclusion

This paper studied trajectory-stealthy physical patch attacks and black-box trajectory anomaly detection for VLA models. We proposed TrajCloak, which combines representation disruption, trajectory deviation constraints, smoothness regularization, adaptive weighting, and black–white–gray visual projection to balance attack effectiveness and trajectory stealthiness. We also introduced TrajGuard, a motion-only detector that measures trajectory abnormality with respect to the clean behavior distribution. Experiments on LIBERO show that TrajCloak lowers Detection Score, Increase, and detection TPR under fixed FPR

thresholds while maintaining high Failure Rate. These results highlight the importance of evaluating VLA security at both the task and trajectory levels.

Acknowledgements. This work was supported in part by the Guangdong Basic and Applied Basic Research Foundation under Grants 2025A1515110166 and 2026A1515010032, and by the China Industry-University-Research Innovation Fund under Grant 2025SE015.

References

1. Athalye, A., Engstrom, L., Ilyas, A., Kwok, K.: Synthesizing robust adversarial examples. In: Proc. ICML. Proc. Mach. Learn. Res., vol. 80, pp. 284–293 (2018)
2. Brohan, A., et al.: RT-1: Robotics transformer for real-world control at scale. In: Robotics: Science and Systems XIX (2023)
3. Chebiyyam, M., Reddy, R.D., Dogra, D.P., Bhaskar, H., Mihaylova, L.: Motion anomaly detection and trajectory analysis in visual surveillance. *Multimedia Tools Appl.* **77**, 16223–16248 (2018)
4. Chirayil Nandakumar, S., Mitchell, D., Erden, M.S., Flynn, D., Lim, T.: Anomaly detection methods in autonomous robotic missions. *Sensors* **24**(4), 1330 (2024)
5. Driess, D., et al.: PaLM-E: An embodied multimodal language model. In: Proc. ICML. Proc. Mach. Learn. Res., vol. 202, pp. 8469–8488 (2023)
6. Eykholt, K., et al.: Robust physical-world attacks on deep learning visual classification. In: Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. pp. 1625–1634 (2018)
7. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. In: Proc. ICLR (2015)
8. Kim, M.J., et al.: OpenVLA: An open-source vision-language-action model. In: Proc. CoRL. Proc. Mach. Learn. Res., vol. 270, pp. 2679–2713 (2025)
9. Liu, B., et al.: LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In: Advances in Neural Information Processing Systems. vol. 36, pp. 44776–44791 (2023)
10. Marcolino, F., Wang, J.: Detecting anomalies in humanoid joint trajectories. In: Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (2013)
11. O’Neill, A., et al.: Open x-embodiment: Robotic learning datasets and RT-X models. In: Proc. IEEE Int. Conf. Robot. Autom. pp. 6892–6903 (2024)
12. Shukor, M., et al.: SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844* (2025)
13. Thys, S., Van Ranst, W., Goedemé, T.: Fooling automated surveillance cameras: Adversarial patches to attack person detection. In: Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (2019)
14. Wang, T., et al.: Exploring the adversarial vulnerabilities of vision-language-action models in robotics. In: Proc. IEEE/CVF Int. Conf. Comput. Vis. pp. 6948–6958 (2025)
15. Xu, Haochuan, Y.S.K.S.H., et al.: Model-agnostic adversarial attack and defense for vision-language-action models. *arXiv preprint arXiv:2510.13237* (2025)
16. Zitkovich, B., et al.: RT-2: Vision-language-action models transfer web knowledge to robotic control. In: Proc. CoRL. Proc. Mach. Learn. Res., vol. 229, pp. 2165–2183 (2023)